

```

import pandas as pd
import numpy as np
from scipy.stats import zscore
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_absolute_error

df = pd.read_csv("Dataset_Day6.csv")

# Handle Missing Values
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = df[col].fillna(df[col].mode()[0])
    else:
        df[col] = pd.to_numeric(df[col], errors='coerce')

original_rows = df.shape[0]

numeric_cols = df.select_dtypes(include=[np.number]).columns
z_scores = np.abs(zscore(df[numeric_cols]))
filtered_entries = (z_scores < 3).all(axis=1)
df_no_outliers = df[filtered_entries]

rows_removed = original_rows - df_no_outliers.shape[0]
reduction_percent = (rows_removed / original_rows) * 100

if reduction_percent <= 30:
    df_clean = df_no_outliers
    print(f"Outliers removed: {rows_removed} rows ({reduction_percent:.2f}%)")
else:
    df_clean = df
    print(f"Outlier removal skipped (would remove {reduction_percent:.2f}% of data)")

print("Final cleaned data shape:", df_clean.shape)

```

```

(myenv) flora_04@Flora:/mnt/d/python/Day6$ python3 solution.py
Outliers removed: 50 rows (3.97%)
Final cleaned data shape: (1209, 9)

```

Question 2

```
print("Original shape:", df_clean.shape)
print("\nCategorical columns to encode:")
print(df_clean.select_dtypes(include='object').columns.tolist())

df_encoded = pd.get_dummies(df_clean, drop_first=True)

print("\nShape after one-hot encoding:", df_encoded.shape)
print("Columns after encoding:\n", df_encoded.columns.tolist())
```

```
Original shape: (1259, 9)
```

```
Categorical columns to encode:
```

```
['Furnishing', 'Status', 'Transaction', 'Type']
```

```
Shape after one-hot encoding: (1259, 10)
```

```
Columns after encoding:
```

```
['Area', 'BHK', 'Bathroom', 'Parking', 'Price', 'Furnishing_Semi-Furnished', 'Furnishing_Unfurnished', 'Status_Ready_to_move', 'Transaction_Resale', 'Type_Builder_Floor']
```

```
(myenv) flora_04@Flora: /mnt/d/python/Day6$
```

Question 3

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=50)

# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

r2_lr = r2_score(y_test, y_pred_lr)
mae_lr = mean_absolute_error(y_test, y_pred_lr)
adj_r2_lr = 1 - (1 - r2_lr) * (len(y_test) - 1) / (len(y_test) - X_test.shape[1] - 1)

# Ridge Regression
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)

r2_ridge = r2_score(y_test, y_pred_ridge)
mae_ridge = mean_absolute_error(y_test, y_pred_ridge)
adj_r2_ridge = 1 - (1 - r2_ridge) * (len(y_test) - 1) / (len(y_test) - X_test.shape[1] - 1)

# Lasso Regression
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)

r2_lasso = r2_score(y_test, y_pred_lasso)
mae_lasso = mean_absolute_error(y_test, y_pred_lasso)
adj_r2_lasso = 1 - (1 - r2_lasso) * (len(y_test) - 1) / (len(y_test) - X_test.shape[1] - 1)

# Print all results
print("\n ♦ Linear Regression:")
print(f"R² Score      : {r2_lr:.4f}")
print(f"Adjusted R²    : {adj_r2_lr:.4f}")
print(f"MAE (in ₹)     : {mae_lr:.2f}")
print("\n ♦ Ridge Regression:")
print(f"R² Score      : {r2_ridge:.4f}")
print(f"Adjusted R²    : {adj_r2_ridge:.4f}")
print(f"MAE (in ₹)     : {mae_ridge:.2f}")
print("\n ♦ Lasso Regression:")
print(f"R² Score      : {r2_lasso:.4f}")
print(f"Adjusted R²    : {adj_r2_lasso:.4f}")
print(f"MAE (in ₹)     : {mae_lasso:.2f}")
```

```
(In [117]: %run 3_1_Linear_Ridge_Lasso.py
Out[117]:
♦ Linear Regression:
R² Score      : 0.6170
♦ Linear Regression:
R² Score      : 0.6170
Adjusted R²    : 0.6000
MAE (in ₹)    : 4555081.27

♦ Ridge Regression:
R² Score      : 0.6169
Adjusted R²    : 0.5999
MAE (in ₹)    : 4555615.78

♦ Ridge Regression:
R² Score      : 0.6169
Adjusted R²    : 0.5999
MAE (in ₹)    : 4555615.78

♦ Ridge Regression:
R² Score      : 0.6169
Adjusted R²    : 0.5999
MAE (in ₹)    : 4555615.78

Adjusted R²    : 0.5999
MAE (in ₹)    : 4555615.78

MAE (in ₹)    : 4555615.78
```